

一句话

"A particular way of organizing data set into computer systems so that its access is efficient, usually at the cost of sublinear complexity of amortized time."

将数据集组织到计算机系统的一种特定方式，以便能够高效访问数据，通常是以摊销时间复杂度低于线性的代价实现的。

Total Ordered Data Set（全序数据集）

Total Ordered Data Set（全序数据集） 是指其中的每一对元素之间都可以进行比较，并且这种比较关系满足数学上“全序关系”的定义。全序关系保证数据可以按照某种确定的顺序排列，从而支持高效的检索、排序等操作。

一、全序关系（Total Order）定义

设集合 S 上有一个关系 R ，若满足以下四个条件，则称 R 是 S 上的**全序关系（Total Order）**：

- 自反性（Reflexive）**：
对任意 $a \in S$ ，有 $a \leq a$
- 反对称性（Antisymmetric）**：
若 $a \leq b$ 且 $b \leq a$ ，则 $a = b$
- 传递性（Transitive）**：
若 $a \leq b$ 且 $b \leq c$ ，则 $a \leq c$
- 全域性（Totality 或 Comparability）**：
对任意 $a, b \in S$ ，必有 $a \leq b$ 或 $b \leq a$

二、Total Ordered Data Set 的含义

它是一类数据集，集合中的元素可以满足上述的“全序”关系，因此它们之间总可以被比较，并能唯一排序。

举例：

数据集	全序依据
学生学号集合	按数字大小排序
姓名集合	按拼音或字典序排列
电影名称集合	按字母表顺序排列
时间戳列表	按时间早晚排序

例如：

- 学号：202101, 202103, 202102 → 可排序为：202101 < 202102 < 202103
- 姓名：Alice, Bob, Charlie → 可排序为：Alice < Bob < Charlie

三、Total Ordered Data Set 的用途

因为数据可排序，很多经典算法都可以高效应用：

- 二分查找 (Binary Search)**：时间复杂度 $O(\log n)$
- 排序算法**：快速排序、归并排序等
- 有序索引结构**：如 B+ 树、红黑树等都基于全序性质构建

四、实际意义（信息系统/数据库中）

在数据库、信息检索系统中，全序数据可以：

- 建立索引（如按照时间或字典序建立 B 树索引）
- 支持范围查询（如查找学号在 [202100, 202199] 的学生）
- 高效支持排序展示（如按姓名字母升序输出）

五、总结

特性	描述
比较性	任意两元素可比较
稳定性	满足数学全序关系
算法支持	支持高效检索与排序
示例	学号、姓名、日期、编号等

Total Ordered Data Set 就是结构规则、关系明确、支持比较与排序的数据集，是计算和算法的基础之一。

Succinct Data Structures

Succinct data structures（简洁数据结构）是一种在接近信息论最优空间下，还能支持高效操作的数据结构。它们的核心目标是：

在使用尽可能少的空间（接近理论下限）的同时，仍然保持对数据的访问、查询、修改等操作具有低时间复杂度（通常是常数或对数时间）。

一、背景和动机

在传统数据结构中，我们往往追求**时间复杂度最小**，而不特别关注空间使用效率。但在以下场景中，空间变得极其重要：

- 大规模数据处理**（如 Web 图、基因组序列、压缩文本索引）
- 内存受限设备**（如嵌入式系统、移动设备）
- 缓存优化与数据压缩**

因此，研究者提出了一类“空间紧凑而功能强大”的数据结构 —— succinct data structures。

二、定义（数学角度）

设某类对象的集合为 S ，其信息熵下界是存储该类对象所需的最小比特数，记为 $I(n)$ 。

则一个 **succinct data structure** 是在空间复杂度为：

$$I(n) + o(I(n)) \quad \text{bits}$$

下，还能支持高效操作（如查询、修改、插入等）的数据结构。

其中， $o(I(n))$ 是一个低阶函数，远小于 $I(n)$ ，例如 $\log n$ 或 $\log \log n$ 。

三、典型例子

1. 位图（Bit Vector） + Rank/Select

对于一个长度为 n 的位图 $B \in \{0, 1\}^n$ ，succinct data structure 可以在：

- 空间： $n + o(n)$ bit
- 操作：
 - **rank(i)**: 查询 $B[1..i]$ 中有多少个 1
 - **select(k)**: 查询第 k 个 1 出现的位置

这类结构被广泛用于文本索引（如 FM-index）与图压缩。

2. Balanced Parentheses（括号结构）

用于表示树或表达式结构（如 XML/JSON 结构树），能在接近理论空间下支持：

- 找匹配括号（匹配子树）
- 查询父子节点关系
- 子树大小计算等操作

3. Succinct Trees（简洁树）

用于存储树结构，如：

- n 个节点的有序树，最优编码需要约 $2n$ bit
- 使用 Jacobson/Farzan-Munro 等结构可以在 $2n + o(n)$ bit 内支持父节点、子节点、LCA（最近公共祖先）等查询，时间复杂度为 $O(1)$ 或 $O(\log n)$

四、与其他概念的比较

数据结构类型	空间使用	操作效率	特点
传统结构（如链表、树）	较大（通常含大量指针）	快	空间浪费
压缩数据结构	空间极小	不一定支持查询	多用于压缩存储
Succinct Data Structure	接近最优（信息论下界）	快（通常 $O(1)$ ）	既压缩又可操作

五、应用场景

- 文本压缩和全文索引（如 FM-index）
- 大规模图结构（Web 图、社交网络图）
- 基因组存储与查询（如 BWT 结构）
- XML/JSON 树的压缩表示
- 压缩布尔矩阵

六、总结

特性	描述
空间复杂度	接近理论信息熵下界
操作	支持常见查询操作，时间复杂度低
应用	大规模数据、高压缩比需求场景
常见构建方式	基于位图 + 辅助结构、树结构编码等

简洁数据结构在“**压缩存储 + 可操作性**”之间取得了高度平衡，是现代计算中非常重要的一类结构。

heap-like notation for a binary tree

Heap-like notation for a binary tree 是一种用数组（通常从下标 1 开始）来表示**完全二叉树**（**Complete Binary Tree**）的方式，它广泛应用于堆（heap）结构中，也可用于其他二叉树的顺序存储表示。

一、基本思想

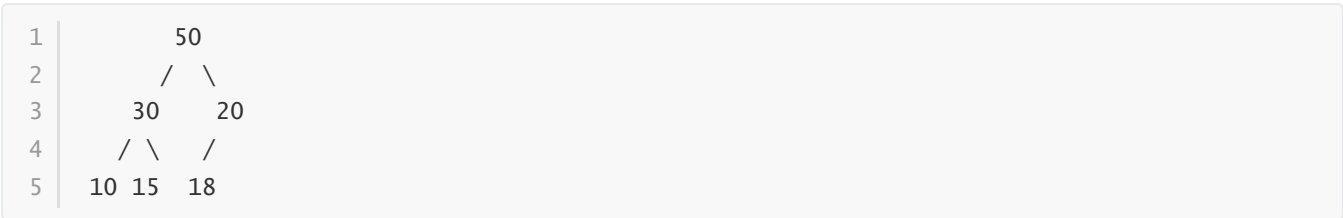
在 **heap-like notation** 中，二叉树的节点被按照**层序遍历**（**Level-order traversal**）的顺序存储到数组中，编号从 **1** 开始（有时也从 0 开始，但典型堆从 1 开始更直观）。

对一个节点编号为 **i**，则其关系如下：

关系	计算公式	含义说明
父节点	$\lfloor i / 2 \rfloor$	第 i 个节点的父节点下标。例如 $i=4$ ，则父为 2。
左子节点	$2 \times i$	第 i 个节点的左子节点下标。例如 $i=2$ ，则左子为 4。
右子节点	$2 \times i + 1$	第 i 个节点的右子节点下标。例如 $i=2$ ，则右子为 5。

二、示例

考虑如下二叉树（以最大堆为例）：



用 heap-like notation 存储为数组（从下标 1 开始）：

下标 (i)	数值
1	50
2	30
3	20
4	10
5	15
6	18

关系验证：

- 节点 1 的左子节点在下标 2（30），右子节点在下标 3（20）
- 节点 2 的左子节点在下标 4（10），右子节点在下标 5（15）
- 节点 3 的左子节点在下标 6（18）

三、优点与限制



优点

- 无需指针，用数组实现树结构
- 适用于完全二叉树，如堆（heap）
- 父子关系计算简单， $O(1)$ 访问

✖ 限制

- 若用于**非完全二叉树**，会造成数组空间浪费
- 不适合表示高度不平衡的树（如链式结构）

四、应用场景

- 堆（Heap）实现
 - 最大堆 / 最小堆
 - 优先队列
- 堆排序（Heap Sort）
- Segment Tree（线段树）
- 二叉索引树（Fenwick Tree）
- 用于某些内存紧凑结构，如堆结构在图算法中的应用（如 Dijkstra）

五、与其它编码方式的比较

编码方式	是否使用指针	空间效率	适用结构
Heap-like notation	否	高	完全/近似完全树
Linked representation	是	灵活	任意结构
Succinct representation	否（位编码）	最优	大规模静态树

Point Query: Min/Max, kth element access

Double-ended Priority Queues

Double-ended Priority Queue（双端优先队列，简称 **DEPQ**）是一种支持在**两端同时进行优先操作**的数据结构，它能够高效地进行以下操作：

一、基本定义

Double-ended Priority Queue（**DEPQ**）是一种特殊的队列，它不仅支持：

- 插入元素
- 删除最小元素（Delete-Min）
- 删除最大元素（Delete-Max）

还支持：

- 获取最小元素（Find-Min）
- 获取最大元素（Find-Max）

与普通优先队列（如堆）相比，DEPQ 允许你同时以优先级的**最大值和最小值**为操作对象。

二、支持的操作

操作名称	描述	时间复杂度（取决于实现）
<code>insert(x)</code>	插入元素 <code>x</code>	$O(\log n)$
<code>find-min()</code>	查找最小元素	$O(1) \sim O(\log n)$
<code>find-max()</code>	查找最大元素	$O(1) \sim O(\log n)$
<code>delete-min()</code>	删除并返回最小元素	$O(\log n)$
<code>delete-max()</code>	删除并返回最大元素	$O(\log n)$

三、常见实现方式

1. Min-Max Heap（最小-最大堆）

- 一种特殊的堆，每一层交替为最小层和最大层。
- 根是最小元素，其孩子是最大元素，其孙子又是最小元素，依此类推。
- 时间复杂度：
 - `insert`： $O(\log n)$
 - `delete-min` / `delete-max`： $O(\log n)$
 - `find-min` / `find-max`： $O(1)$

2. 两个堆（Min Heap + Max Heap）

- 使用一个最小堆和一个最大堆分别维护最小值和最大值，同时使用映射同步两堆中的元素位置。
- 实现复杂度较高，但可以灵活维护。

3. 平衡二叉搜索树（如红黑树、AVL树）

- 同时支持快速插入、删除、最小值、最大值查找。
- 时间复杂度均为 $O(\log n)$ 。
- 可以支持顺序统计（如第 k 小、第 k 大）。

四、应用场景

- 调度系统**：如任务分配中找最优（最大）或最劣（最小）任务。
- 金融系统**：动态维护最高出价和最低出价。
- 多窗口优先级处理**：如操作系统中资源调度的双端限制。
- 滑动窗口问题**：动态维护最大最小值（结合双端队列或树结构）。

Min-Max Heap

什么是Min-Max Heap

Min-Max Heap 是一种基于数组的完全二叉树（Complete Binary Tree），它将节点按照深度分为 **Min 层** 和 **Max 层**，如下规则：

- 根节点在 **Min 层**（深度为0），表示整棵树的最小值。
- 它的孩子在 **Max 层**（深度为1），表示子树最大值。
- 孙子节点回到 **Min 层**（深度为2），表示子子树最小值。
- 如此交替。

也就是说：

- 偶数层（0, 2, 4, ...）是 **Min 层**，维护最小值；
- 奇数层（1, 3, 5, ...）是 **Max 层**，维护最大值。

操作

操作	时间复杂度	说明
<code>findMin()</code>	$O(1)$	根节点
<code>findMax()</code>	$O(1)$	根的两个子节点中较大者
<code>insert(x)</code>	$O(\log n)$	插入并“上浮”到正确位置
<code>deleteMin()</code>	$O(\log n)$	删除根节点并“下沉”
<code>deleteMax()</code>	$O(\log n)$	删除最大节点并“下沉”

1. 节点层级判定

给定节点在数组中的下标 `i`（从1开始），其层数为：

$$\text{level}(i) = \lfloor \log_2(i) \rfloor$$

判断是否在 Min 层：

$$\text{is_min_level}(i) = \text{level}(i) \% 2 == 0$$

2. 插入操作

- 将新元素插入数组尾部；
- 判断其所在层（Min 层或 Max 层）；
- 比较并向上浮动到正确的位置，分别遵循 Min Heap 或 Max Heap 的规则。

3. 删除操作（查找类似，就是没有删除操作）

`deleteMin()`

- 删除根节点（下标1）；
- 用数组最后一个元素替换；
- 向下“下沉”，保持 Min-Max Heap 性质。

`deleteMax()`

- 查找根的两个子节点（下标2、3）中较大的那个；
- 删除之，用最后一个元素替换；
- 向下“下沉”。

动态顺序统计

常见操作

1. **OS-SELECT(i, S):**

- 这个操作返回动态集合 S 中第 i 小的元素。
- 例如，如果 $i=3$ ，那么操作将返回集合中第三小的元素。

2. **OS-RANK(x, S):**

- 这个操作返回元素 x 在集合 S 中排序后的秩（即位置）。
- 秩是指元素在排序后的集合中的位置，例如，如果 x 是集合中第二小的元素，那么它的秩就是 2。

增强数据结构的想法

- **使用红黑树:**
 - 红黑树是一种自平衡的二叉查找树，它保证了树的高度大致为 $O(\log n)$ ，其中 n 是树中节点的数量。
 - 红黑树的这种特性使得它在插入、删除和查找操作中都能保持较好的性能。
- **存储子树大小:**
 - 在每个节点中，除了存储键值（key）外，还存储该节点的子树大小（size）。
 - 子树大小是指以该节点为根的子树中节点的数量。
 - 存储子树大小可以使得在进行顺序统计操作时，能够快速计算出某个元素的秩或者找到第 i 小的元素。

节点表示

- 幻灯片中展示了节点的表示方式，其中包含两个主要部分：
 - **key:** 节点的键值，即存储在该节点的实际数据。
 - **size:** 以该节点为根的子树的大小。

通过这种增强，可以在对数时间内完成动态顺序统计操作，这对于需要频繁进行此类操作的应用场景非常有用。

操作：

操作	方法说明	时间复杂度
查找最小值 (min)	从根节点沿左子树向下遍历，直到最左节点，即为最小值节点。	$O(\log n)$
查找最大值 (max)	从根节点沿右子树向下遍历，直到最右节点，即为最大值节点。	$O(\log n)$
查找第 i 小元素	以当前节点为根，比较 i 与左子树大小 left_size： • 若 $i = \text{left_size} + 1$ ，当前节点即为目标节点 • 若 $i \leq \text{left_size}$ ，递归左子树查找 • 否则递归右子树查找 $i - \text{left_size} - 1$	$O(\log n)$
查找第 i 大元素	转化为查找第 $(n - i + 1)$ 小元素，其中 n 为树中节点总数。	$O(\log n)$

冯·诺依曼体系结构（Von Neumann Architecture）

冯·诺依曼体系结构（Von Neumann Architecture）是现代计算机系统的基础架构模型，由数学家约翰·冯·诺依曼（John von Neumann）于1945年提出。它规定了计算机的基本组成部分及其工作方式。下面将对该体系结构进行详细讲解，并标注相关名词的英文术语。

一、冯·诺依曼体系结构的五大基本组成部分

冯·诺依曼体系结构包括以下五个主要部分：

中文名称	英文术语	功能说明	常见设备举例（含英文）
运算器	Arithmetic Logic Unit (ALU)	执行各种算术运算（如加、减、乘、除）和逻辑运算（如与 AND、或 OR、非 NOT）	通常集成在 中央处理器 CPU（Central Processing Unit） 内部
控制器	Control Unit (CU)	负责从存储器中取指、解释指令并协调指令的执行，控制其他部件有序工作	同样集成在 CPU 内部，与 ALU 协同工作
存储器	Memory / Main Memory	用于存储指令和数据，是 CPU 可直接访问的高速存储区域	- 主存： RAM（Random Access Memory） - ROM（只读存储器， Read-Only Memory ） - Cache Memory（缓存）

中文名称	英文术语	功能说明	常见设备举例（含英文）
		存储器可细分为主存与辅存：主存为 CPU 直接访问；辅存用于长期数据保存	- 辅存（非主存）： • 硬盘 HDD (Hard Disk Drive) • 固态硬盘 SSD (Solid State Drive) • U 盘 USB Flash Drive • 光盘 CD/DVD (Compact Disc / Digital Versatile Disc)
输入设备	Input Device	用于向计算机输入信息、指令或控制信号	- 键盘 Keyboard - 鼠标 Mouse - 摄像头 Camera / Webcam - 麦克风 Microphone - 扫描仪 Scanner
输出设备	Output Device	将计算机处理结果以人可理解的形式呈现，如文字、图像、声音等	- 显示器 Monitor / Screen - 打印机 Printer - 音响 Speaker - 耳机 Headphone

二、冯·诺依曼体系的核心思想

1. 存储程序控制（Stored Program Concept）

程序和数据**统一存储**在主存中，计算机按照指令顺序逐条读取和执行。

在此之前，早期计算机需要人工手动接线或更换控制面板来“更改程序”，效率极低。

2. 指令顺序执行（Sequential Instruction Execution）

CPU 从内存中**按顺序**读取指令，执行完一条指令后自动跳转到下一条，除非遇到跳转指令（如 `goto` 或 `if-else`）。

3. 数据与指令存储在同一个存储器中（Unified Memory for Data and Instructions）

程序指令和数据**共用同一存储区域**，这既简化了硬件结构，也带来了潜在问题（如程序代码可被修改，需保护机制）。

三、工作流程：指令周期（Instruction Cycle）

一条指令的执行通常包括以下几个步骤：

- 取指（Fetch）**：从内存中取出下一条指令（由程序计数器PC指定地址）。
- 译码（Decode）**：控制器对指令进行分析，确定要执行的操作和操作数位置。
- 执行（Execute）**：运算器或其他单元执行指令操作。
- 存储/写回（Store）**：将结果写回内存或寄存器。
- 更新程序计数器（PC）**：指向下一条指令地址。

这个过程被称为 **取指-译码-执行** 循环（Fetch-Decode-Execute Cycle）。

四、典型结构图示意（文字版）



五、优点与局限

优点：

- 简化了计算机设计结构，易于硬件实现
- 统一的存储器模型，使程序可以灵活地读写数据
- 成为现代计算机发展的基础架构

局限性（也称为“冯·诺依曼瓶颈”Von Neumann Bottleneck）：

- 指令与数据共享同一内存总线，导致读取速度受限（不能同时读取指令和数据）
- 指令按顺序执行，不适合并行处理任务
- 缓存未命中会严重拖慢执行速度

六、现代发展（补充）

虽然今天的计算机架构仍以冯·诺依曼为基础，但在实际中已引入了许多优化技术，例如：

- **哈佛架构 (Harvard Architecture)**：指令和数据分开存储与处理，缓解瓶颈
- **流水线 (Pipelining)**：多个指令阶段并行处理，提高吞吐率
- **缓存 (Cache Memory)**：在 CPU 内部使用小容量高速缓存减少访问延迟

七、总结对照表

组成部件	英文术语	功能说明
运算器	Arithmetic Logic Unit	运算逻辑操作
控制器	Control Unit	指令控制，协调各部件工作
存储器	Memory	存储数据和程序指令
输入设备	Input Device	数据输入接口
输出设备	Output Device	数据输出接口
程序计数器	Program Counter (PC)	指向下一条指令地址
指令周期	Instruction Cycle	取指 → 译码 → 执行 → 写回
冯诺依曼瓶颈	Von Neumann Bottleneck	受限于指令/数据共享总线的处理速度限制

外排序：External Sorting. Improve Run Gen.Improve Run Merging

摊还分析 (Amortized Analysis)

摊还分析是一种用于分析 **一系列操作平均性能** 的方法。
它不关注单个操作的最坏情况，而是分析一整个操作序列的 **平均成本 (Amortized Cost)**。

一、聚合方法 (Aggregate Method)

原理：

假设执行 (n) 个操作，总成本为 (T(n))，则每个操作的摊还成本为：

$$\frac{T(n)}{n}$$

即将所有操作作为一个整体进行分析，计算平均代价。

特点：

- 简单直观；
- 不考虑操作之间的差异；
- 适用于操作代价变化不大或变化可预测的情形。

示例：

动态数组插入操作中，大多数插入的代价为 (O(1))，但当数组扩容时，插入代价为 (O(n))。聚合方法通过分析所有插入操作的总成本，然后除以插入次数，得到摊还成本。

二、记账方法 (Accounting Method)

原理：

- 给每个操作分配一个摊还成本 (Amortized Cost)；
- 多余的代价存为 **余额 (Credit)**，用于支付后续更昂贵的操作；
- 保证总摊还成本 ≥ 实际总成本。

策略：

- **便宜的操作：** 摊还成本 > 实际成本，多出来的部分作为余额；
- **昂贵的操作：** 摊还成本 < 实际成本，用之前积累的余额补贴。

特点：

- 更精细地控制成本分配；
- 适用于部分操作成本极高、但能由其他操作“补贴”的场景。

示例：

动态数组中每次插入分配 3 元作为摊还成本：

- 实际只花 1 元插入；
- 剩余 2 元作为余额，用于未来扩容操作。

三、潜能方法 (Potential Method)

原理：

为数据结构定义一个 **势能函数 (Potential Function)** (Φ)，表示当前状态下的“预存资源”。

第 (i) 次操作的摊还成本为：

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

其中：

- (c_i)：第 (i) 次操作的实际成本；
- ($\Phi(D_i)$)：第 (i) 次操作后的势能；
- ($\Phi(D_{i-1})$)：操作前的势能；
- ($\hat{c}_i$)：摊还成本。

解释：

- 如果势能增加，说明本次操作为后续做了准备；
- 如果势能减少，说明用掉了之前积累的资源。

特点：

- 数学形式化，表达灵活严谨；
- 非常适合状态动态变化频繁的数据结构分析。

四、三种方法对比

方法	英文名称	分析粒度	是否为每个操作分配摊还成本	特点
聚合方法	Aggregate Method	所有操作整体	否	简单直观，适用于操作成本规律明显的场景
记账方法	Accounting Method	每个操作	是	有余额机制，适合个别操作较昂贵的情况
潜能方法	Potential Method	每个操作	是	利用势能函数建模，适合复杂状态的数据结构

红黑树

红黑树（Red-Black Tree）是一种自平衡的二叉查找树（Binary Search Tree, BST），它通过在每个节点中增加一个颜色位（红色或黑色），来确保树在执行插入、删除等操作后，始终保持“近似平衡”，从而保证所有基本操作的时间复杂度为 $O(\log n)$ 。

一、红黑树的基本特点

红黑树是在二叉查找树的基础上，通过引入“颜色”和严格的规则来控制树的高度。

每个节点包含：

- key：键值
- color：颜色（红色 Red 或 黑色 Black）
- left：左子节点
- right：右子节点
- parent：父节点

二、红黑树的五条性质（保持平衡的关键）

1. 每个节点要么是红色，要么是黑色。
2. 根节点是黑色的。
3. 每个叶子节点（NIL节点，空节点）是黑色的。
4. 红色节点不能连续出现（红色节点的子节点一定是黑色的）。
5. 从任一节点到其所有后代叶子节点的路径都包含相同数目的黑色节点（黑高一致）。

三、红黑树的平衡特性

这些规则确保了以下性质：

- 红黑树的最长路径不会超过最短路径的2倍。
- 树的高度最多为 $2 \log(n + 1)$ ，保持了**对数高度**，从而实现 $O(\log n)$ 的操作效率。

四、红黑树的常见操作时间复杂度

操作	时间复杂度	说明
搜索	$O(\log n)$	类似普通BST搜索
插入	$O(\log n)$	插入后通过颜色变换和旋转调整
删除	$O(\log n)$	删除后通过颜色变换和旋转调整

五、红黑树如何保持平衡？

关键操作：旋转（Rotate）+ 颜色变换

- 左旋（Left Rotate）** 和 **右旋（Right Rotate）**：用于调整树的结构。
- 颜色翻转（Recoloring）**：调整节点的颜色以满足红黑树的性质。

在插入或删除节点后，若违反了红黑树的性质，则通过“旋转 + 颜色变换”恢复平衡。

六、红黑树的应用

红黑树被广泛应用于许多标准库和系统中，例如：

- C++ STL 的 `map` 和 `set`
- Java 的 `TreeMap` 和 `TreeSet`
- Linux内核中的进程调度、内存管理等模块
- 数据库索引系统（如PostgreSQL）中某些数据结构

七、与其他平衡树的对比

树类型	平衡性	插入删除代价	常用场景
红黑树	中等	$O(\log n)$ ，性能稳定	多用于标准库和系统编程
AVL树	较严格	插入/删除代价稍高	查找频繁、插入少的场景
Treap（随机树）	概率平衡	插入删除平均 $O(\log n)$	竞赛或研究中常见
B树/B+树	多路平衡树，适合磁盘	插入删除较复杂	数据库与文件系统

